



WEEK 3 - PYTHON PROGRAMMING

Loops

Repetition, accumulation and nested loops

Session 1 - 120 minutes, including a break

Instructor: MinGi KYUNG

Learning Goals & Agenda

- Repeat tasks with `for`, `range()` and `while`.
- Trace accumulators and explain when a loop stops.
- Use `break` and `continue` deliberately.
- Build multiplication tables and star patterns with nested loops.

Review 10 min / `for` 25 min / `while` 20 min / `Break` 10 min

Nested loops 15 min / Guided labs 30 min / Review 10 min

Review: One Student's Result

- An if statement selects which block runs.
- A score of 60 or higher is a pass.
- How could we process 30 students?

```
score = 75

if score >= 60:
    print("Pass")
else:
    print("Fail")

# Output: Pass
```

for: One Value at a Time

- Place the sequence after in.
- Each iteration assigns the next value to the variable.
- A colon and four-space indentation define the body.
- The loop ends after the last value.

```
for letter in "ABC":  
    print(letter)  
  
print("Done")
```

```
# Output  
# A  
# B  
# C  
# Done
```

Integer Ranges with range()

- range(stop) starts at 0.
- range(start, stop) includes start.
- The stop value is always excluded.
- A range represents an integer sequence. It is not a list.

```
for i in range(3):  
    print(i)  
# 0, 1, 2 on separate lines  
  
for i in range(1, 4):  
    print(i)  
# 1, 2, 3 on separate lines
```

Steps & Reverse Ranges

- The third argument is the step.
- Use a negative step to count down.
- A mismatched direction gives an empty range.
- A step of zero raises `ValueError`.

```
for i in range(2, 7, 2):  
    print(i)  
# 2, 4, 6  
  
for i in range(3, 0, -1):  
    print(i)  
# 3, 2, 1  
  
for i in range(3, 0):  
    print(i)  
# No output
```

Accumulating a Total

- Initialize total before the loop.
- `total += n` means `total = total + n`.
- Print the final result after the loop.
- What happens if `total = 0` is inside the loop?

```
total = 0

for n in range(1, 6):
    total += n

print(total)
# 15
```

Tracing the Accumulator

- Predict each variable's value before running the code.
- Each iteration uses the previous total.

Step	n	Before	After
1	1	0	1
2	2	1	3
3	3	3	6
4	4	6	10
5	5	10	15

Practice 1: Sum of Even Numbers

- Iterate over the integers from 1 to 10.
- Use if and % to identify even numbers.
- Add only even numbers to total.
- Expected result: 30. Work for 5 minutes.

pass is a placeholder statement that does nothing.

```
total = 0

for n in range(1, 11):
    # Add your condition and update here.
    pass

print(total)
```

while: Repeat While True

- Test the condition before running the body.
- If it starts false, the body runs zero times.
- Update the value that controls termination.

```
count = 1

while count <= 3:
    print(count)
    count += 1

print("Done", count)
# 1
# 2
# 3
# Done 4
```

Execution Order & Infinite Loops

- Initialize, check the condition, run the body, then check again.
- Without `count += 1`, this condition stays true.
- Stop a running loop with Ctrl+C in a terminal or the Stop button in Colab.

count	count <= 3	Action
1	True	Print 1
2	True	Print 2
3	True	Print 3
4	False	Exit loop

break: Exit the Loop

- Repeat input until the user requests an exit.
- `break` exits the nearest enclosing loop.
- When the input is `q`, skip the echo and leave the loop.

```
while True:
    text = input("Message (q: quit): ")
    if text == "q":
        break
    print("Input:", text)

print("Done")
```

continue: Skip This Iteration

- Skip the remaining statements in the current iteration.
- Continue with the next iteration.
- In a while loop, do not accidentally skip the update needed to stop.

```
for n in range(1, 6):  
    if n == 3:  
        continue  
    print(n)
```

```
# Output  
# 1  
# 2  
# 4  
# 5
```

Choosing for or while

- To process each character in a string: for.
- To repeat a known number of times: for with range().
- To repeat while a condition holds: while.
- Both loop types can contain if statements.

Choose a loop and explain why:

1. Print each character in a word.
2. Read messages until the user enters q.
3. Print the 7-times table from 1 to 9.

Practice 2: Sum Until Zero

- Read one integer per iteration.
- Stop on 0 and print the total.
- Check the stop value before adding.
- Assume integer-formatted input. Work for 5 minutes.

Hints: total = 0, while True, int(input()), break.

Input sequence	Expected total
3, 5, 0	8
0	0
-2, 5, 0	3

Break & Midpoint Check

10-minute break

- What is the last value in `range(1, 5)`?
- When does `while` test its condition?
- How do `break` and `continue` differ?

Next: repeating a loop inside another loop.

Nested Loops

- The outer loop chooses a group.
- For each outer value, the inner loop runs through all of its values.
- The example prints six pairs.
- Indentation determines which loop owns a statement.

```
for row in range(1, 3):  
    for col in range(1, 4):  
        print(row, col)
```

```
# 1 1  
# 1 2  
# 1 3  
# 2 1  
# 2 2  
# 2 3
```

Tracing Nested Loops

- Finish the inner loop before advancing the outer loop.
- For each new row, col starts at 1 again.
- Two rows and three columns produce six body executions.

Outer row	Inner col values
1	1, 2, 3
2	1, 2, 3

After both loops finish:
row = 2
col = 3

Lab 3A: One Multiplication Table

- Create multiplication.py.
- Read an integer from 2 to 9.
- For valid input, print nine equations using factors 1 through 9.
- Otherwise, print "Invalid table" and skip the single-table section.

Assume integer-formatted input. Use an f-string for each equation.

```
Input: 7
First line: 7 x 1 = 7
Last line: 7 x 9 = 63
Number of equations: 9
```

```
Input: 1
Invalid table
```

Lab 3B: Tables from 2 to 9

- Extend multiplication.py to print all tables after the single-table result.
- The outer loop chooses the table.
- The inner loop chooses the factor.
- Print a blank line after each table.

Place the all-tables extension after the input-validation block so it always runs.

```
for table in range(2, 10):  
    for factor in range(1, 10):  
        # Print one equation here.  
        pass  
    print()
```

Lab 3 Checks

- Test 2 and 9 as valid boundary inputs.
- Test 1 and 10 as invalid inputs.
- The all-tables section must contain 72 equations: 8 tables times 9 factors.
- Check the first and last equations.

All-tables section

First: $2 \times 1 = 2$

Last: $9 \times 9 = 81$

A valid single-table input adds
9 equations before this section.

An invalid input adds none.

Star Patterns: Rows & Columns

- Use one outer iteration for each row.
- Row 1 contains one star, row 2 contains two, and so on.
- `print("*", end="")` keeps output on the same line.
- `print()` ends the current row.

```
for row in range(1, 4):  
    for col in range(row):  
        print("*", end="")  
    print()
```

```
# *  
# **  
# ***
```

Lab 4: A Growing Triangle

- Create stars.py and read an integer height from 1 to 10.
- Print a left-aligned triangle using nested loops.
- Print "Invalid height" outside the allowed range.
- Extension: print a shrinking triangle after a blank line.

Use nested loops for the main task. String repetition is an alternative to discuss later.

Input: 4

```
*  
**  
***  
****
```

Optional extension

```
****  
***  
**  
*
```

Lab 4 Checks

- Height 1 produces exactly one row with one star.
- Height 4 produces four rows and ten stars in total.
- Height 10 produces ten rows.
- Heights 0 and 11 produce only the invalid-height message.

Common mistakes

`range(1, height)` misses the last row.

`print()` inside the inner loop
puts each star on its own line.

`range(height)` in the inner loop
creates a rectangle.

Common Loop Mistakes

- `range(1, 10)` includes 1 through 9, not 10.
- Initialize an accumulator before the loop.
- Keep the final summary outside the loop.
- A while loop needs a condition change or an exit.
- Put the row-ending `print()` after the inner loop.

Before running: predict the first iteration, the last iteration and the final output.

When debugging: print the loop variable and accumulator temporarily.

Next Session: Functions

- Give a reusable task a name with `def`.
- Pass values into a function through parameters.
- Return a result for the caller to use.
- Turn a loop-based calculation into a reusable function.

Preparation: identify the input and output of your even-number sum program.

Lists, tuples, dictionaries and sets will follow in Week 4.